SellerShield AI member 3 complete guide

Backend and rules engine from product idea to working demo

Prepared for the backend owner on 7 November 2026

You build the server that connects payment records, customer messages, shipping labels and the merchant dashboard. Your code compares addresses, applies explicit rules, stores the evidence behind each result and returns a decision the merchant can understand. Your first milestone is a stored order that produces an explained BLOCKED result through the API.

Read sections 1 to 5 to understand the product and your tools. Follow sections 6 to 13 while working on the backend. Sections 14 to 20 cover testing, teammate integration, planned extensions, deployment and the demo. The appendices provide a glossary, troubleshooting steps and references.

The project documents define the team responsibilities. The implementation described here is the single-merchant sandbox backend in the Sentrix workspace. Features marked planned need further work before the team can demonstrate them as working integrations.

1 The product in plain language

A customer pays through PayPal using shipping address A. Later, they ask support to ship to address B. A seller might accept the request without checking how it affects the original payment transaction. SellerShield compares the payment address with the intended shipment destination before dispatch.

For example, the payment says 123 Main Street, Apartment 4, Austin, Texas 78701. The shipping label says 987 Main Street at the same city and ZIP code. SellerShield identifies the changed house number and tells the merchant to hold dispatch.

The product has a practical purpose even when the customer made an honest mistake. An address-change request is not proof of fraud. Your system detects a destination mismatch or uncertainty and helps the merchant review it.

PayPal's US Seller Protection policy requires shipment to the shipping address on the transaction details page, along with other conditions. Address alignment is one check, not a guarantee that a future dispute will be decided in the seller's favor. The dashboard's PROTECTED label is SellerShield's local rule result. See reference 3 for the policy.

The intended merchant experience is to see a queue of orders, open an order, compare the addresses, read the reasons, then choose an action. Later features help prepare dispute evidence and perform merchant-approved PayPal actions.

What the three results mean

Result	Meaning	Merchant response
PROTECTED	Addresses agree under the supported rules and configured checks pass	Continue through the merchant's fulfillment workflow
REVIEW	Data, extraction, eligibility or comparison needs a human check	Resolve the uncertainty before dispatch
BLOCKED	Reliable inputs show a meaningful destination mismatch	Hold dispatch and review remediation

BLOCKED changes this application's workflow. It does not freeze PayPal funds or physically stop a carrier. Preventing shipping outside the dashboard would require integration with the merchant's fulfillment system.

2 Your responsibilities and team boundaries

The research PDF calls Member 3 the Backend and Rules Engine Lead. It assigns you FastAPI routing, address normalization, field-level differences and SQLite audit persistence. The architecture plan adds orchestration, evidence bundling and backend deployment. The current implementation uses Postgres for local development and deployment, with SQLite in the test suite.

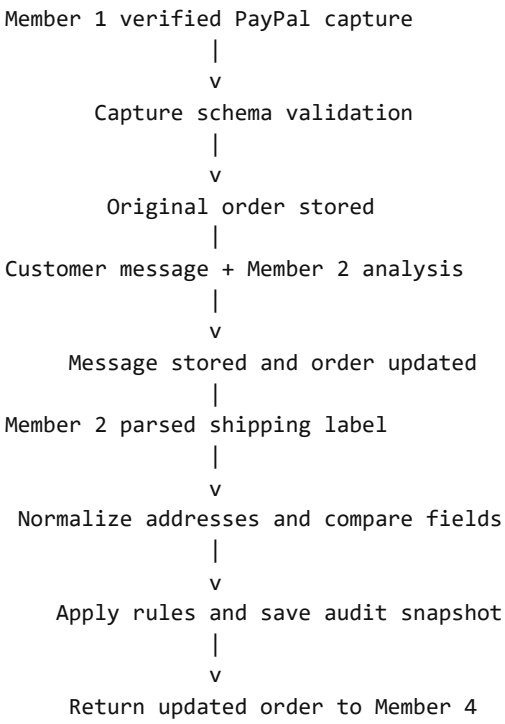
Member	Owns	What you exchange
1 PayPal lead	Webhook verification, trusted PayPal fetches, payment and dispute API calls	They supply normalized captures and action outcomes; you supply persistence and workflow integration
2 AI lead	Message intent extraction, label vision and factual evidence summaries	They supply validated analysis and parsed labels; you supply stored inputs and contracts
3 You	Backend app, schemas, database, comparison, verdicts, audit history and deployment	You connect the modules and return consistent results
4 Frontend lead	Queue, order drawer, address difference display and action UI	You supply endpoint schemas, examples, authentication behavior and errors
5 Demo and QA lead	Fixtures, rehearsals, documentation, pitch and video	You supply stable scenarios, reset instructions and clear integration status

Use one FastAPI application with modules owned by different members. Separate ownership does not require separate servers. Member 1 can mount a verified webhook router in your app, and Member 2 can expose functions or a router within the same backend.

You coordinate the handoffs. Member 1 remains responsible for PayPal protocol details; Member 2 remains responsible for the extraction model and prompts. You remain responsible for validating their outputs at your boundary and for the decisions your service records.

3 The full flow through your backend

The essential flow is payment data, optional message analysis, parsed label, address comparison, verdict, database commit, dashboard response.



Before a label exists, a message can set MESSAGE_FLAGGED when it requests an address change. The current backend does not compare the proposed message address to decide dispatch. Final address evaluation uses the PayPal shipping address and the latest parsed label. Message intent confidence can still cause REVIEW after a label exists.

The order of arrival can vary. A label can arrive before a message. A message can first arrive without analysis and receive analysis later. When the latest relevant inputs change, the service recomputes the result and preserves a new verdict snapshot.

Three different addresses have three different meanings. The PayPal address is the original transaction reference. The proposed address is what the customer requested in a message. The label address is where the shipment is intended to go. Keep these separate in names, storage and explanations.

4 What exists and what remains planned

Capability	Current workspace status
Order persistence through a shared ingestion function	Implemented for trusted normalized captures
PayPal webhook signature verification and live sandbox fetches	Member 1 integration still required
Queue and order detail APIs	Implemented
Message intake with optional structured analysis	Implemented
Direct LLM calls and inbox connections	Member 2 integration still required
Parsed label JSON intake	Implemented
Image upload and vision parsing	Member 2 integration still required

Capability	Current workspace status
Address comparison and three verdicts	Implemented for the supported US forms
Audit input snapshots and rule version	Implemented in the database
Public audit history API	Not implemented
Merchant override or review resolution	Not implemented
Refund, reorder and tracking submission	Not implemented in this backend slice
Dispute evidence bundle and submission	Planned extension
Authentication	HTTP Basic for a single-merchant sandbox demo
Multi-merchant accounts and tenant isolation	Not implemented
Render deployment configuration	Present; successful deployment must be checked separately

An endpoint returning a local result is not proof that PayPal accepted an action. Keep fixture, sandbox and future production data clearly identified. The current schema permits fixture and sandbox capture sources only.

5 Your actual technology stack

Tool	Your use	File or location
Python 3.12	Backend language and supported runtime target	.python-version and render.yaml
FastAPI	HTTP routes, dependencies, errors and generated API schema	app/main.py
Pydantic	Data types, field constraints and boundary validation	app/schemas.py
Uvicorn	Server process that runs the FastAPI application	README.md and render.yaml
SQLAlchemy 2	Database models, queries and transactions	app/db.py and app/service.py
psycopg 3	Python driver connecting SQLAlchemy to Postgres	requirements.txt
Postgres 16 locally	Durable relational database	compose.yaml
Alembic	Versioned database schema changes	migrations and alembic.ini
usaddress	Parsing US street addresses into components	app/rules.py
RapidFuzz	Similarity signal for street or city text	app/rules.py

Tool	Your use	File or location
pytest and HTTPX	Rule tests and FastAPI TestClient support	tests
Docker Compose	Runs the local Postgres service	compose.yaml
Render	Hosts the planned web service and managed Postgres	render.yaml

FastAPI handles requests. Uvicorn runs the server process. Pydantic checks payload shapes. SQLAlchemy reads and writes records. Postgres stores those records. Alembic creates or changes the tables. These are different jobs within the same backend.

usaddress identifies components probabilistically. It cannot confirm that an address exists and does not normalize it for you. Your code adds suffix and direction mappings, preserves important components and routes unsupported or ambiguous cases to REVIEW. RapidFuzz provides string similarity, not a shipping authorization.

The current project already has SQLAlchemy and Alembic. Keep those choices rather than introducing SQLModel or another database layer. No queue, Redis service, vector database or separate model-training pipeline is needed for the current slice.

6 The repository reading order

Read the files in this order so each layer has context.

Order	File	What to understand
1	README.md	Setup, integrations, limitations and deployment procedure
2	fixtures/scenarios.py	Concrete capture, message and label examples
3	app/schemas.py	Accepted fields, types, required values and response structure
4	app/rules.py	Normalization, comparisons, uncertainty and verdict precedence
5	app/db.py	Tables and database connection setup
6	app/service.py	Ingestion, latest input selection, recomputation and view assembly
7	app/main.py	Routes, transactions, authentication, CORS and error responses
8	app/seed.py	Repeatable synthetic demo records
9	tests/test_rules.py	Examples of intended decision behavior
10	tests/test_api.py and tests/conftest.py	API, persistence, concurrency and test database lifecycle
11	migrations and render.yaml	Schema creation and hosted deployment

Keep changes in the layer that owns the behavior. Change field constraints in schemas, comparison policy in rules, cross-record workflows in service, HTTP handling in main and schema changes in migrations. A rule change should not require copying comparison logic into routes or the frontend.

7 Set up and run locally

Use Python 3.12, Docker Desktop and a terminal in the project root. The following commands are for PowerShell. Read the commands before running them; they install dependencies, start a local database and insert synthetic demo orders.

Create the Python environment

```
Set-Location 'C:\Users\Hp\Documents\Sentrix'
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python --version
python -m pip install -r requirements.txt
```

If the py launcher is unavailable, use the path to a Python 3.12 executable. A different system Python does not establish compatibility with the project's 3.12 target. If PowerShell blocks activation, call .\.venv\Scripts\python.exe and other executables by their full environment paths instead of changing machine-wide policy.

Start the database and set configuration

```
docker compose up -d --wait
$env:DATABASE_URL = 'postgresql+psycopg://sellershield:local-only@localhost:5432/sellershield'
$env:MERCHANT_USERNAME = 'merchant'
$env:MERCHANT_PASSWORD = 'replace-with-your-own-long-random-demo-password'
$env:CORS_ORIGINS = 'http://localhost:5173'
alembic upgrade head
python -m app.seed
uvicorn app.main:app --host 127.0.0.1 --port 8000 --no-access-log
```

The database password local-only is the local Docker example value, not a credential to reuse on a hosted database. Choose a private merchant password. Avoid placing credentials in screenshots, committed files or a public frontend build.

The app reads process environment variables. Copying .env.example to .env does not load it automatically. Set variables in the shell that starts the app or configure the host's environment settings.

Docker Compose binds Postgres to the local machine and uses a named volume. Stopping containers normally preserves that volume. Avoid commands that remove volumes unless you intend to discard the local database.

Confirm the backend is reachable

Open http://127.0.0.1:8000/docs in a browser. Authorize with the configured merchant username and password. GET /health/ready should return a ready response. GET /orders requires authentication and should show the seeded scenarios.

```
Invoke-RestMethod 'http://127.0.0.1:8000/health/ready'
```

The health endpoint checks database connectivity. It does not verify the full application schema, PayPal connectivity or AI availability. Run migrations before treating the service as usable.

8 Understand the shared contracts

app/schemas.py is the source of truth for requests and responses. The base contract rejects unknown fields. A typo such as confidence_score in place of confidence should fail validation rather than silently disappear.

The current address schema supports country US. Amounts use USD and positive decimal strings with two fractional digits. State format and ZIP validity are checked during address normalization. Some address fields accept empty strings at the schema level so incomplete data can be preserved and routed to REVIEW.

Capture from Member 1

```
{
  "order_id": "DEMO-CLEAN",
  "paypal_order_id": "FIXTURE-ORDER-clean",
  "capture_id": "FIXTURE-CAPTURE-clean",
  "amount": {"value": "149.99", "currency": "USD"},
  "paypal_address": {
    "line1": "123 Main Street",
    "line2": "Apt 4",
    "city": "Austin",
    "state": "TX",
    "postal_code": "78701",
    "country": "US"
  },
  "seller_protection": "ELIGIBLE",
  "source": "fixture"
}
```

This is a synthetic example. Actual sandbox captures use source sandbox and values fetched and verified by Member 1. Capture is a shared internal contract, not a public unverified HTTP intake endpoint.

Message with Member 2 analysis

```
{
  "order_id": "DEMO-CLEAN",
  "id": "MSG-guide-1",
  "text": "Please ship to my office instead",
  "ai_analysis": {
    "intent": "ADDRESS_CHANGE_REQUEST",
    "confidence": 0.98,
    "quote": "ship to my office instead",
    "proposed_address": null
  }
}
```

Supported intent values are ADDRESS_CHANGE_REQUEST, DELIVERY_QUESTION, THANKS, OTHER and AMBIGUOUS. The quote must be a nonempty exact substring of the original message. This confirms the quoted text came from that message; it does not independently prove the intent interpretation is correct.

Analysis may be absent initially. The same message ID and unchanged text may receive analysis later. Reusing an existing ID with conflicting data produces a conflict instead of overwriting history silently.

Parsed label from Member 2

```
{
  "order_id": "DEMO-CLEAN",
  "address": {
    "line1": "987 Main Street",
    "line2": "Apt 4",
    "city": "Austin",
    "state": "TX",
    "postal_code": "78701",
    "country": "US"
  },
  "tracking_number": "SIMULATED-TRACKING",
  "carrier": "SIMULATED",
  "confidence": 0.98
}
```

POST /labels accepts this parsed JSON. It does not accept an image file. Member 2 must implement upload and vision parsing or an adapter that feeds this contract. Missing label confidence routes to REVIEW.

Reading the response

An OrderView includes capture information, state, timestamps, latest message, latest analysis, latest label, comparison, verdict and recommended_action. Before a label is evaluated, verdict and recommended_action are null; comparison.match is also null. Null means no complete result yet, not a green result.

The same response shape is returned by queue, detail, message and label routes. Member 4 can update the drawer directly from a successful message or label response, then refresh the queue as needed.

9 The address engine from input to verdict

The public entry point in app/rules.py is evaluate(inputs). It reads the PayPal address, the label address, optional intent analysis and seller-protection status. It returns comparison, verdict and recommended action without writing to the database.

normalize(address) first validates the address shape. It requires line1, city, state and postal code to contain text. It accepts known two-letter state or territory codes and either a five-digit ZIP or ZIP plus four extension. It then parses the street and unit through usaddress.

The supported parser result must be a Street Address with a house number and street name. Unsupported parsed components, repeated labels and incomplete unit information produce an uncertain result. PO boxes are currently routed to REVIEW instead of being supported automatically.

The code canonicalizes common suffixes such as Street and St, and direction words such as North and N. It compares house number, assembled street, unit identifier, city, state and full postal code. Country is fixed to US by the schema.

Match describes the comparison facts

Comparison value	Current meaning
MATCH	Supported normalized fields agree and the original address objects are equal
NEAR_MATCH	Normalized fields agree but the original address objects differ

Comparison value	Current meaning
MISMATCH	At least one compared normalized field differs
null	Parsing or completeness prevents a reliable comparison

The full ZIP extension is preserved. A five-digit ZIP and the same ZIP with a four-digit extension are not automatically treated as equivalent by the current code. Missing unit information on one address versus a present unit on the other currently produces a unit difference; with otherwise reliable inputs it can produce BLOCKED. These are current behaviors to discuss with evaluated fixtures before changing policy.

Verdict applies uncertainty before mismatch

The current precedence is explicit:

```
Any configured uncertainty -> REVIEW
Otherwise MISMATCH         -> BLOCKED
Otherwise                  -> PROTECTED
```

Uncertainty includes incomplete or unsupported addresses, missing label confidence, label confidence below 0.90, intent confidence below 0.90, AMBIGUOUS intent or a seller-protection status other than ELIGIBLE. A confidence of exactly 0.90 meets the current threshold.

PARTIALLY_ELIGIBLE and UNKNOWN both route to REVIEW in this implementation. They do not automatically pass. This is the configured local policy, not a statement that every partially eligible PayPal transaction lacks all protection.

A street or city difference with a RapidFuzz ratio of at least 85 also causes REVIEW. The code treats similar wording as a reason for human judgment, not proof of equivalence. These numeric cutoffs are local heuristics that need evaluation on fixtures.

Examples of the actual rules

Inputs assuming other checks pass	Match	Verdict
Same complete address	MATCH	PROTECTED
Main Street versus Main St	NEAR_MATCH	PROTECTED
Apt 4 versus number sign 4	NEAR_MATCH	PROTECTED
House number 123 versus 987	MISMATCH	BLOCKED
Apt 4 versus Apt 9	MISMATCH	BLOCKED
Austin versus Houston	MISMATCH	BLOCKED
Main versus similar spelling Mane	MISMATCH	REVIEW
Different ZIP and label confidence 0.65	MISMATCH	REVIEW
Missing street or unsupported PO box	null	REVIEW

A MISMATCH with REVIEW is not a contradiction. The comparison records the difference, while the verdict says human checking is required because another condition is uncertain. Preserve the reasons for both.

The current comparison does not use the proposed message address as an independent destination check. An unrecognized or untranslated customer request can therefore require additional workflow design. Document this limitation in integration discussions instead of describing the prototype as a complete fraud detector.

Before changing suffix mappings, numeric cutoffs, missing-unit behavior or precedence, add representative test cases, agree on the intended result and bump the rule version when the policy changes. Keep the original inputs in the audit snapshot.

10 Database and audit history

app/db.py defines four SQLAlchemy models. The initial Alembic migration creates the same four tables.

Table	Important data	Why you need it
orders	IDs, capture ID, original address, decimal amount, eligibility, source, state and timestamps	Preserves the transaction reference and current workflow state
messages	Order link, external message ID, original text and optional analysis	Links customer communication to the order
labels	Order link and parsed label JSON	Preserves each supplied label
verdicts	Order link, timestamp, rule version, input snapshot and result	Explains what the engine decided from a particular set of inputs

Capture IDs are unique. Message external IDs are unique within an order. Labels and verdicts append as new records. The API has no route to edit a saved verdict snapshot, although database administrators can still change stored data. An application audit table is not cryptographic tamper-proof storage.

Amounts are stored as decimal numeric values and returned as two-decimal strings. Keep currency with the amount. Avoid binary floating point when handling payments.

Transactions keep related changes together

The request-scoped database dependency begins a transaction. A successful operation commits when the transaction exits. An exception rolls it back. A label insert, verdict insert and order-state update should succeed together or leave no partial update.

flush sends pending changes to the database inside the current transaction. commit makes the transaction durable. The shared ingest_capture helper flushes but leaves the commit to its caller. Member 1 must call it inside an appropriate transaction.

Message and label routes lock the order row before modifying inputs. On Postgres, this serializes concurrent changes to the same order so two requests do not compute results from inconsistent input sets. SQLite tests do not establish that the Postgres row-locking path works.

Latest inputs and the current view

latest selects the record with the greatest internal ID for the requested order. view assembles the order and latest message, label and verdict. recompute updates timestamps, handles the no-label state and otherwise evaluates the latest inputs before appending a verdict.

This is a simple latest-message policy. A newer unrelated message can become the message used in the current view. If the product must retain every unresolved address-change request, extend this behavior deliberately and test message ordering; do not assume the prototype already does that.

11 API behavior and error handling

Route	Auth	Current behavior
GET /health/ready	Public	Checks database connection and returns ready or 503
GET /orders	Merchant	Returns newest-first full order views
GET /orders/{order_id}	Merchant	Returns one order view or 404
POST /messages	Merchant	Stores a message or attaches delayed analysis, recomputes and returns the order
POST /labels	Merchant	Appends parsed label JSON, recomputes and returns the order
GET /docs	Public documentation page	Exposes API descriptions; merchant requests still require authentication

The current queue returns all order views. It has no pagination or server-side filtering. That is sufficient for a small seeded demo. Add pagination when real queue size justifies it and agree on a new response shape with Member 4.

Errors the frontend should handle

HTTP status	Meaning here	Frontend behavior
401	Missing or incorrect merchant credentials	Request valid session credentials
404	Unknown order	Show an order-not-found message
409	Existing capture or message identifier conflicts with new data	Explain the conflict; do not silently overwrite
422	Request violates the contract or exact-quote rule	Show the relevant validation issue
500	Unexpected backend operation failed	Show failure and offer a controlled retry
503	Database unavailable on readiness check	Treat the backend as unavailable

The app removes rejected input values from validation error responses so buyer details are not echoed unnecessarily. Request logs use route templates instead of order IDs and omit bodies. Uvicorn access logging is disabled in the configured command.

Authentication uses HTTP Basic for one merchant. It needs HTTPS when used remotely. CORS lets the configured browser origin send permitted requests; it does not authenticate a merchant and does not provide tenant isolation.

12 A complete local API walkthrough

Use synthetic records for this exercise. Run the setup and seed steps first. Seeding inserts DEMO-CLEAN, DEMO-MISMATCH and DEMO-UNCERTAIN and does not overwrite existing orders.

Observe the three seeded outcomes

In /docs, authorize and call GET /orders. You should find one PROTECTED, one BLOCKED and one REVIEW order when the seed database is fresh. The synthetic tracking and source fields must stay labeled as simulated or fixture data.

Open GET /orders/DEMO-MISMATCH. Compare paypal_address.line1 with label.address.line1. The house number changes from 123 to 987. The comparison should report MISMATCH and a reason describing a different number.

Open GET /orders/DEMO-UNCERTAIN. Its label confidence is 0.65. Even if the address agrees, the verdict should be REVIEW because confidence is below 0.90.

Create a new mismatch through the label route

Call POST /labels with the parsed-label example from section 8 for DEMO-CLEAN. It changes that fixture order's latest label to 987 Main Street. The response should show MISMATCH, BLOCKED and HOLD_REFUND_REORDER if the other current inputs remain eligible and confident.

Then post the original 123 Main Street address with confidence 0.98. The result should return to PROTECTED, and the database should retain both verdict snapshots. There is currently no public history endpoint, so inspect those records through your tests or database tools.

Exercise human review

Post another label with the original address and confidence 0.65. The verdict should be REVIEW. Repeat with confidence omitted and expect REVIEW again. Do not convert missing confidence to an artificial 1.0 in an adapter.

Exercise validation and conflicts

Try confidence 9 and expect 422. Try an unknown order ID and expect 404. Send a quote that is not present in the message and expect 422. Reuse a stored message ID with different text while retaining a valid matching quote and expect 409.

These calls change the fixture orders. The seed command will not reset them because it preserves existing orders. For a pristine recording, use a new disposable demo database or a separately designed, reviewed reset procedure. Do not delete shared records casually.

13 Your daily development workflow

Begin with the demo path and current contract. Pick one observable outcome, such as making a particular valid formatting variant produce NEAR_MATCH. Read the shared function and its callers before editing it.

Add a focused test when the behavior is nontrivial. Make the change in the owning layer, run the relevant tests, then run the backend suite. If the contract changes, update schemas, fixtures, examples and affected frontend or integration callers together.

Re-run one meaningful API scenario after changes to orchestration, serialization or persistence. A passing pure rule test does not prove the response model or transaction path works. A passing SQLite API test does not prove the Postgres concurrency path works.

Keep source code changes separate from deployment and data operations. Review migrations before applying them to a shared database. Never remove an original transaction address to accommodate a customer's new request.

At the end of the day, show the team a working scenario, report remaining integration boundaries and provide exact payload examples. A short demonstration of an explained verdict is more useful than reporting that a module is almost done.

Build order if you are starting from scratch

1. Agree US scope, JSON contracts, null handling and rule precedence.
2. Create the FastAPI app, database models and migration.
3. Ingest one synthetic order through the shared helper.
4. Accept parsed label JSON and implement the plain comparison function.
5. Persist the verdict and return the order through the API.
6. Cover formatting variants, destination changes and uncertainty with tests.
7. Connect Member 1 and Member 2 to the same contracts.
8. Connect Member 4 to queue, detail and mutation responses.
9. Verify Postgres behavior, deploy and rehearse the actual demo.
10. Add approved actions and evidence only after the core path is stable.

14 Testing without destroying the wrong database

The suite contains direct rule tests and API tests. The API fixture runs against SQLite and optionally Postgres. Read tests/conftest.py before setting TEST_DATABASE_URL.

```
python -m pytest -q
```

Without TEST_DATABASE_URL, Postgres variants skip. A green run with skipped Postgres tests is not evidence that hosted Postgres behavior or row locking was checked.

The Postgres test fixture runs migrations and deletes all rows in the four application tables after each test. TEST_DATABASE_URL must point to a separate, disposable test database. Never use the local working demo, a teammate's shared database or the deployed database for this variable.

With the local Compose database running, create a separate database, then opt in:

```
docker compose exec postgres createdb -U sellershield sellershield_test
$env:TEST_DATABASE_URL = 'postgresql+psycopg://sellershield:local-only@localhost:5432/sellershield_test'
python -m pytest -q
```

If the test database already exists, reuse it only when it is disposable and not shared. Remove the test variable from a shell after testing when appropriate:

```
Remove-Item Env:TEST_DATABASE_URL
```

What the current tests cover

Area	Examples
Address rules	Exact match, abbreviation, unit formatting, different house, unit, ZIP, city and state
Uncertainty	Low or missing label confidence, low intent confidence, ambiguous intent and unconfirmed eligibility
API boundary	Authentication, unknown orders, out-of-range confidence and invented quotes
Persistence	Order survives app recreation; verdict includes inputs and rule version
Transactions	Failed recomputation rolls back the label and state change
Retries	Identical capture accepted; conflicting capture rejected
Message evolution	Delayed analysis accepted; conflicting original text rejected
Seed data	Running the seed twice does not duplicate the three orders
Concurrency	Postgres-only test checks serialized message and label updates

Additional useful cases include missing units, ZIP extensions, multiple messages, out-of-order analysis, multiple labels and disconnects during external actions. Add tests when implementing those behaviors or resolving their policy. Do not claim they are covered by a neighboring test.

15 Integrating Member 1 and Member 2

Member 1 supplies trusted capture records

Member 1 should verify the PayPal webhook signature, fetch trusted order or capture information, map it to Capture and call ingest_capture. The helper must not be exposed through an unauthenticated public route that accepts a claimed PayPal address.

The existing integration shape is:

```
from app.db import session_factory
from app.schemas import Capture
from app.service import ingest_capture

sessions = session_factory()

with sessions.begin() as db:
    capture = Capture.model_validate(normalized_record)
    order = ingest_capture(db, capture)
```

normalized_record must come from Member 1's verified workflow. Identical capture retries return the existing order. Conflicting capture data or identifiers raise Conflict. The caller owns the transaction commit.

If the verified webhook router is mounted inside this FastAPI app, it should use PayPal's verification process rather than the merchant Basic-auth dependency. Agree on error mapping and webhook acknowledgement behavior with Member 1.

Member 2 supplies extracted facts

Give Member 2 the schemas and exact examples. They should return supported intent values, a source quote, a confidence within 0 to 1 and an optional proposed US address. The label parser should return the complete destination components, tracking number, carrier and confidence when available.

Agree whether Member 2 calls internal functions, mounts a router or submits to the existing protected intake routes. The current backend accepts analysis; it does not invoke an LLM or a vision model itself. Describe the adapter explicitly so neither teammate waits for the other to implement it.

If extraction fails, preserve an understandable failure and hold for review. Do not manufacture a plausible address or a high confidence just to make the contract pass. Customer text and label content are untrusted input, including instructions embedded within them.

Your handoff to Member 4

Provide the API base URL, /docs location, successful and failing payloads, null-result behavior, source marker and error statuses. Explain that /labels takes parsed JSON, not multipart image data. Agree where the image-upload UI will call Member 2's implementation.

The frontend should display backend verdicts and reasons. It should not recompute policy in JavaScript. Use runtime credentials for the single-merchant demo and avoid committing them or embedding a shared password into a public build.

16 Planned merchant actions and evidence

This section explains later work from the architecture plan. These capabilities are not implemented by the current routes.

Refunds tracking and replacement orders

Your role is to validate the merchant request, check order state, coordinate Member 1's integration and record the result. Member 1 implements PayPal calls and protocol details. Add an action record and clear states when this work begins; the existing four tables do not represent a complete financial-action history.

For an action, authorize the merchant on the server, identify the exact order and current inputs, record the request and pass a stable retry identifier to the integration where supported. Only record success after a confirmed successful result. A timeout can mean outcome unknown; reconcile it before repeating a financial operation.

Creating a reorder link is different from receiving a paid replacement order. Entering tracking is different from PayPal accepting tracking. A local BLOCKED result is different from a refund. Show these states separately.

Human review also needs a designed API. The expanded state diagram allows review decisions, but the current implementation has no merchant override route or approval record. Store who approved what, when and against which input version before describing that feature as complete.

Evidence preparation and submission

Build a stable snapshot of original transaction data, relevant customer messages, label information, comparisons, verdicts, action history and available tracking facts. Member 2 can draft a summary from those stored facts. Avoid adding claims that are absent from the records.

Serialize the saved bundle deterministically and compute SHA-256 over its exact bytes using hashlib. Retain the same bytes or a durable version identifier for later checking. A hash detects a change relative to that saved value; it does not authenticate the original transaction or stop someone from replacing both bundle and hash.

Separate preview from submission. Record merchant approval for the exact evidence version. If the pack changes, require review of the changed pack. Member 1 handles the provide-evidence request and response. Mark EVIDENCE_SUBMITTED only after the submission is confirmed, not merely after pack generation.

A boolean approved supplied by the browser is insufficient by itself. The backend must authenticate the merchant, check authority over the order and bind approval to the actual action and version.

17 Deploying the backend

render.yaml defines a Python web service and a managed Postgres database in the same region. The checked-in configuration selects paid service plans. Review the host's displayed resources and charges before creating them. Having a YAML file does not mean those resources are already deployed.

Use the Git repository and branch agreed by the team. If the workspace is only a folder, create or connect the intended repository before configuring the host. Avoid assuming a remote exists because source files are present.

The deployment sequence is dependency installation, database migration, then application startup. The configured start command binds to the host-provided PORT on all interfaces and disables access logging.

```
Build       pip install -r requirements.txt
Pre deploy  alembic upgrade head
Start       uvicorn app.main:app --host 0.0.0.0
            --port $PORT --no-access-log --log-level info
```

The displayed start command wraps for readability; render.yaml contains it on one line.

Set MERCHANT_USERNAME, MERCHANT_PASSWORD and the actual frontend CORS_ORIGINS in the hosting configuration. The blueprint injects the database connection string. Do not put live secrets into source control or use the local-only database password remotely.

Deployment checks

1. Confirm dependency installation and migration complete successfully.
2. Call /health/ready and confirm the service reaches Postgres.
3. Request /orders without credentials and expect 401.
4. Authenticate and check queue and detail responses.
5. Insert only synthetic scenarios for the demo when desired.
6. Run the frontend against the deployed backend and verify CORS and auth.
7. Recreate or restart the app and confirm stored order data remains.
8. Exercise one message and label flow and inspect the saved verdict.

Postgres preserves structured records independently of the web process. If image upload is added later, local web-service files are not automatically durable. Choose a persistent file-storage location and store its reference in the database.

Keep production or real customer data outside the public hackathon demo. The current app has one merchant credential pair and no tenant isolation, account lifecycle or production payment workflow.

18 The demo you should make reliable

Your backend demo should prove that an input reaches the rules, an explained verdict reaches the frontend and the database retains the decision. The current seeded version can demonstrate all three local verdicts. A real PayPal sandbox demonstration additionally needs Member 1's verified integration.

Start with DEMO-CLEAN and show the original address, matching label and PROTECTED result. Then open DEMO-MISMATCH and show the changed house number and BLOCKED reason. Finally, open DEMO-UNCERTAIN and show why the lower confidence requires REVIEW.

For a live walkthrough, send a new message and parsed label through the intake routes, refresh the dashboard and show the new result. Identify the message source and parsed label as simulated until the corresponding integrations run. Avoid calling a fixture capture a PayPal response.

Keep a known fixture set and a separate demo database. Prepare endpoint examples in advance, confirm credentials privately and rehearse with the actual deployed frontend. Do not install dependencies or change the schema during the recording.

Definition of done for your current slice

- The documented Python 3.12 setup runs for another team member.
- Migrations create the expected tables on a fresh Postgres database.
- Trusted capture ingestion preserves the original address and handles identical retries.
- The queue and detail routes return the shared shape with correct null handling.
- Parsed labels produce explained, persisted verdicts.
- Message updates recompute as documented and preserve earlier verdict snapshots.
- Invalid requests and unauthorized requests fail clearly.
- SQLite tests pass and Postgres checks are reported separately.
- The frontend can use the hosted backend and display the three scenarios.
- Planned PayPal actions and evidence are identified as pending until implemented and checked.

19 A practical work plan

Use the original plan's sequence even when dates move. Do not start by polishing evidence summaries while the mismatch path cannot reach the dashboard.

Milestone	Your deliverable	Completion check
Contracts and app skeleton	Shared schemas, app, database configuration and initial migration	Teammates can validate the same example records
Core path	Capture helper, label intake, comparison, verdict and order API	A mismatched label produces BLOCKED and saved reasons
Address behavior	Normalization rules and uncertainty precedence	Representative equivalence and mismatch tests pass

Milestone	Your deliverable	Completion check
Team integration	Real sandbox capture and actual AI outputs use existing contracts	One complete order flow works without manual database edits
Deployment	Hosted API, persistent Postgres, migration and private config	Frontend can complete the flow after an app restart
Optional extensions	Approved actions and evidence workflow	Real outcomes are recorded and distinguishable from previews
Freeze and recording	Stable synthetic scenarios and rehearsed demo	Recording path runs repeatedly without new code changes

At each check-in, report the working outcome, the next integration you need and one concrete blocker if there is one. For example, say that parsed-label evaluation works but no image adapter is connected, rather than saying that the AI flow is done.

20 Common mistakes to catch before handoff

Do not overwrite paypal_address with the proposed address. That destroys the reference needed for comparison. Keep a replacement order as a separately identified transaction.

Do not interpret a high fuzzy score as destination equality. House numbers, units and directional components need explicit attention. A single character can identify a different apartment or street.

Do not treat REVIEW as a minor warning that permits automatic dispatch. It is the path for uncertainty and unconfirmed eligibility. Frontend actions should reflect that workflow.

Do not describe the current backend as reading images or calling the LLM. It accepts structured outputs. Keep that boundary visible in team integration notes.

Do not treat a seed command as a reset command. Existing orders are left intact. Use separate disposable databases for testing and clean recording fixtures.

Do not report all database behavior as checked when the Postgres variants skipped. Record what environment ran and what remains untested.

Do not infer action success from a button click or an HTTP timeout. Save confirmed outcomes and reconcile uncertain external operations.

Do not expand the scope to multiple payment providers, carrier rerouting or multi-tenant accounts until the agreed hackathon path is reliable. Those features require additional data contracts and security boundaries.

Appendix A Troubleshooting

Symptom	First checks
Python command uses the wrong version	Run python --version and use the environment created with Python 3.12
Activation is blocked	Invoke the environment's executables directly
Docker connection fails	Start Docker Desktop, then inspect docker compose ps

Symptom	First checks
Port 5432 is occupied	Check for another local Postgres; agree a port change and update DATABASE_URL consistently
Missing table error	Confirm the database URL and run alembic upgrade head for the intended development database
App fails at startup	Confirm MERCHANT_USERNAME, MERCHANT_PASSWORD and DATABASE_URL are set in that shell
Browser returns 401	Check runtime Basic credentials; documentation access alone does not authorize API calls
Browser reports CORS failure	Match CORS_ORIGINS to the actual frontend origin including scheme and port
Label request returns 422	Send parsed JSON with the exact schema, not an image or renamed fields
Message request returns 422	Check confidence range, enum value and whether quote occurs exactly in text
Message request returns 409	Check whether that message ID already has different text or analysis
Obvious mismatch returns REVIEW	Read all reasons; uncertainty takes precedence in the current engine
Fixture does not reset after seeding	Seed preserves existing orders; use an isolated fresh demo database
Tests skip Postgres	Set TEST_DATABASE_URL only to a disposable test database
Health passes but orders fail	Health checks connectivity only; confirm migrations and application requests

Appendix B Glossary

Term	Meaning in this project
Backend	Server code that validates requests, applies rules and stores data
Endpoint	A method and URL exposed by the backend
JSON contract	Agreed fields, types and meanings exchanged between modules
Webhook	A service's HTTP notification about an event
Capture	The payment capture record linked to an order
Normalization	Converting supported formatting variants into comparable components
Deterministic rule	Code that returns the same result for the same inputs and rule version
Verdict	SellerShield's workflow decision after comparison and uncertainty checks
ORM	A library mapping Python model objects to database tables
Migration	A versioned change to the database schema
Transaction	A group of database changes committed or rolled back together
Idempotency	Safe handling of repeated requests without repeating the intended effect
Row lock	Database coordination that prevents concurrent changes from racing on one row

Term	Meaning in this project
Audit snapshot	Saved inputs and result for a specific evaluation
Fixture	Synthetic data used in tests or a demonstration
Sandbox	Provider test environment for integrations
CORS	Browser rules governing cross-origin access; not authentication
SHA-256	Hash function producing a digest of exact bytes

Appendix C References and team decisions

1. SellerShield AI Research and Roadmap, supplied PDF, especially pages 4 and 5 for architecture and member responsibilities.
2. SellerShield AI Architecture and Member Plan, supplied Markdown file, especially module responsibilities, contracts and Member 3's plan.
3. PayPal US Seller Protection policy: https://www.paypal.com/us/legalhub/paypal/seller-protection
4. FastAPI request bodies: https://fastapi.tiangolo.com/tutorial/body/
5. FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/
6. FastAPI server operation: https://fastapi.tiangolo.com/deployment/manually/
7. SQLAlchemy session and transaction basics: https://docs.sqlalchemy.org/en/20/orm/session_basics.html
8. Alembic tutorial: https://alembic.sqlalchemy.org/en/latest/tutorial.html
9. usaddress project documentation: https://github.com/datamade/usaddress
10. RapidFuzz scoring documentation: https://rapidfuzz.github.io/RapidFuzz/Usage/fuzz.html
11. Docker Compose quickstart: https://docs.docker.com/compose/gettingstarted/
12. Render Blueprints: https://render.com/docs/infrastructure-as-code
13. Render persistent storage behavior: https://render.com/docs/disks
14. Current workspace files: README.md, requirements.txt, app, fixtures, tests, migrations, compose.yaml and render.yaml.

Before extending the prototype, agree on missing-unit handling, ZIP extensions, supported address forms, latest-message policy, merchant review APIs and the exact ownership of image upload. The documents and current code do not settle every product decision. Keep the team contract and evaluated examples updated when those decisions change.